InkZero AR — Project Proposal

HCC 552, Fall 2026. Arthur Bowman. Solo.

Motivation

Disney Lorcana is a two-player trading card game with decks of 60 or more cards and hidden hands. Good players tend to be predictable because they play meta decks, the decks that are winning tournaments right now, so the hidden information at the table is more knowable than it looks. New players often lose on cognitive load more than strategy, that is, on how many questions they have to hold in their head at once. What is legal? What is under threat? What was I shown and have forgotten? What does the opponent likely hold? Digital card games can give players trackers for those. The physical table gives them little, and the tournament rules allow no notes during a match other than lore totals (Ravensburger, Tournament Rules 7.14.2026, section 5.2). The opponent's cards also face the opponent, so every one of them is upside down from the player's seat, and reading one means turning it over in your head or asking. Anything that pulls a player's eyes off the other player can cost them information, so the help should live where the player is already looking.

I play competitive Lorcana at tournaments and at local game stores in the region. A practice tool that helps a player see the hidden information at the table would be valuable to me. I asked other players if they would try one out, and they said yes.

InkZero is a project I conceived and have led since before I joined the M.S. in Artificial Intelligence at Grand Valley State University. I named it in my application as work I wanted to do here. It has three parts: a recognition part, an engine part, and an AR part. This course is the place to build the AR part, so this proposal is the AR part.

AR research question

How should hidden-information help be shown on a passthrough headset, an augmented-reality (AR) headset whose cameras show the real room with graphics drawn on top, during live play of a physical card game, so that the player can play a normal game?

Human-computer interaction (HCI) work is design, implementation and evaluation. This project covers design and implementation: a built system and a comparison against stated criteria. Evaluation with users is outside this course's scope, so it is future work.

Three design considerations follow from that question:

- The overlay is passive. During play, the player does not touch it.
- Each overlay reads at a glance.
- The player can keep their eyes on the opponent, so they miss less from them.

Related work and the research gap

From my research, nobody I found has shown hidden-information help for a physical card game on a passthrough headset. The closest related work is in the table below, which gives for each the game, the device, who it was for, what is missing for this problem, and what I take from it.

Work	Game	Device	For whom	What is missing	What I take
cARcassonne (Kadish et al., 2022)	Carcassonne, a tile board game	HoloLens 2 headset	Novice players	The board and tiles were virtual, tiles were drawn from a shared stack, and no player held private information	A model that scores likely placements for a novice player in a headset. The scores were not shown to the player, only whether a placement was legal
PokAR (Gamba and Monroy-Hernández, 2023)	Heads-up Texas hold'em, two-player poker, with physical cards	A phone running a Snapchat lens, a camera filter	The players at the table	Showed live chip counts and action buttons, and left odds as future work	Its most experienced players rated the chips, buttons and messages as getting in the way, which is why the overlay here is passive and small
TCG-AR (Cioppa et al., 2026)	The Pokémon card game, with physical cards	One overhead camera, extra side cameras, and a computer	Stream viewers through broadcast software, not the players	Offers no play advice; face-down cards are read only as card backs, and the operator keys in hidden values by hand	The card-recognition path for future work
Studierstube Mah-Jongg (Szalavári et al., 1998)	Mahjong with virtual tiles	See-through headsets	The players	Virtual tiles, no likelihood readout	Help information rendered only in that player's own view, which is the privacy model for the overlay
Official Lorcana Challenge stream overlay (Ravensburger, 2026)	Lorcana, physical cards	Overhead camera and a stream graphic	Viewers	Shows lore, player names and ink colors in a fixed frame beside the mat; tracks nothing hidden	A fixed panel beside the mat is already how this game is shown to a 2D audience; the panel placement asks whether the same shape works as a 3D object beside a real mat

I searched arXiv, the ACM and IEEE libraries, Google Patents, the app stores and GitHub, and I asked players in the competitive scene.

I framed the project as a design question so it stays open if more prior work surfaces. My background is in design, animation and brand management, so the overlay design is a problem I can bring experience to.

The new part is the display: showing a player what they have seen and what the odds are, while they play a normal game.

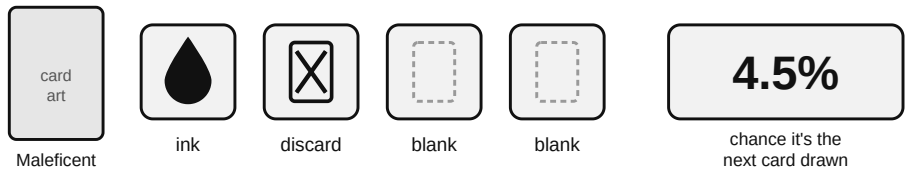
Proposed solution

The question is how hidden-information help should be shown on a passthrough headset so that the player can play a normal game. This part of the project builds the display. The engine belongs to another part. Here the engine is kept simple on purpose, so the display question is the one being answered.

Section overview, by part:

1. The display in one picture
2. Where the display sits
3. What each row shows
4. What I track to fill the rows
5. How it gets drawn
6. The demo

1. The display in one picture. The player wears a passthrough headset and plays a normal game. For every card in the opponent's deck, the headset shows a row: a few small marks that say where each copy of that card has gone, and a number that says how likely that card is to come off the top of the deck next. The rows update on their own as the game moves.




```
# Sample panel row. In this scene 44 of the opponent's cards are unseen (38 in the deck, 6 in hand),
  and 2 of this card's 4 copies are among them. 2 ÷ 44 = 4.5%, the chance it is the next card drawn.
  On the mat the card is right there, so the thumbnail is left off.
```

Figure 1. one panel row. A card thumbnail on the left, four pips showing ink, discard, blank, blank, and the next-draw percentage beside them. The note under it works the 4.5% example: 2 unseen copies over 44 unseen cards, 38 in the deck and 6 in hand.

The labels in the figures are a legend for the reader. Ideally the row shows the icons and the number only. The goal is icons clear enough that a player learns the five once and does not have to think about them again. If an icon is hard to remember, the fix is a better icon, or a label on it, and the readability check is where that shows up.

2. Where the display sits. AR tends to put information in one of two places: on the thing it is about, or beside it in a fixed spot. The first is called situated (Willett et al., 2017); the second is a heads-up display, a HUD, the form streams and games already use. Each has a known strength: situated help is easy to tie to its card, and a HUD is familiar and can show everything at once. I build both and compare them:

- On the mat. Each row is drawn under the physical card it is about, anchored to the mat, and facing the player even though the card faces the opponent. The five counts sit in one strip on the opponent's near mat edge. Only cards in play have a place for a row, so this placement shows less than the panel does. That is part of the comparison.
- On a panel. The rows for every card and the five counts sit in one fixed panel off to the side of the play area, and the cards and mat are left clean.

Both are passive and both are private to the player wearing the headset.

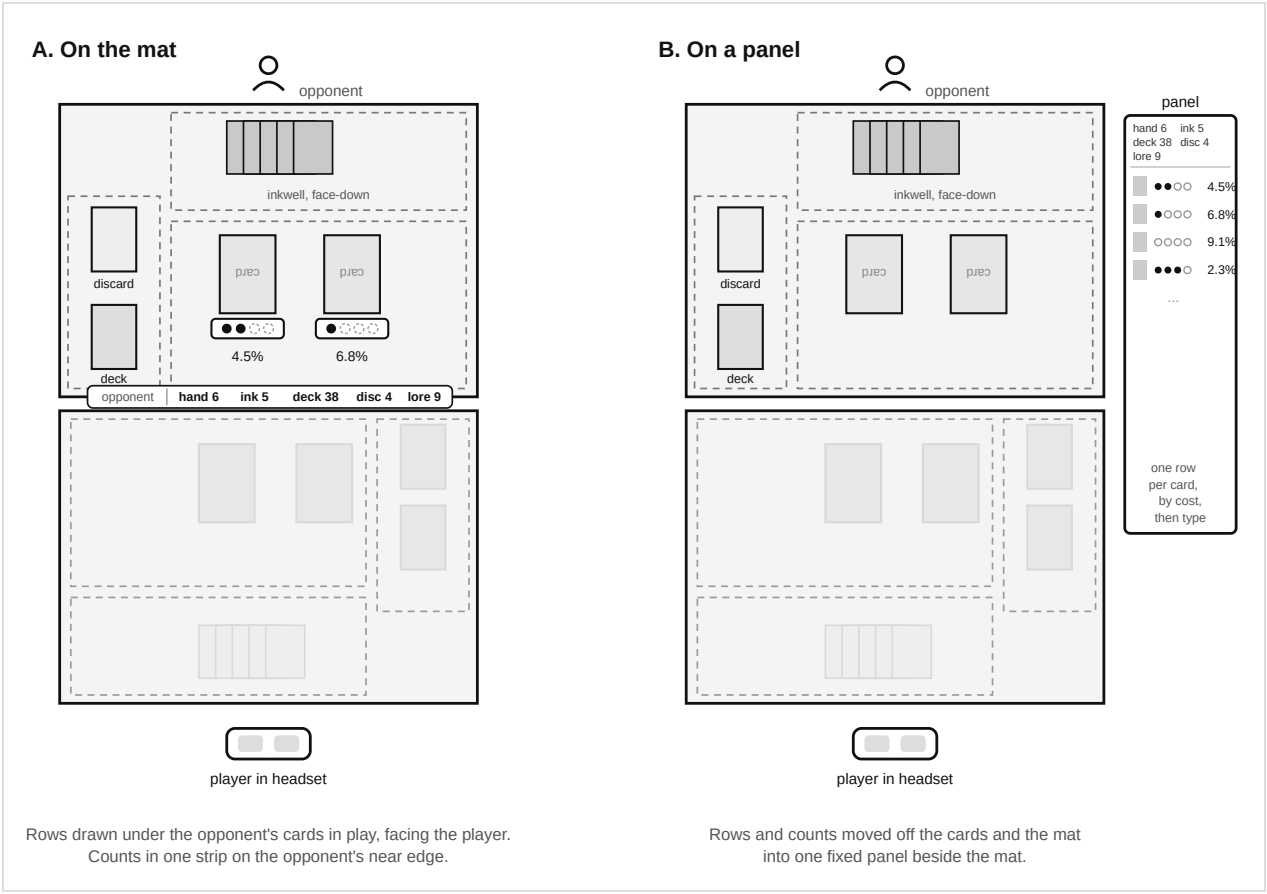


Figure 2. the two placements, drawn top-down with the player's own side faded. A, on the mat: rows under the opponent's cards in play, counts strip on the seam between the two mats. B, on a panel: rows and counts in a panel beside the mat.

3. What each row shows. Every card in the opponent's deck list gets a row, because which cards matter is the player's call, not the tool's. Each row has two parts.

- The pips. One small mark per copy in the deck list, up to four, each showing where that copy is. Lorcana players already see decks this way: the deck-building site Dreamborn shows each card as a stack of its copies with a count marker, such as 4x, and its list view shows one row per card. The pips keep that shape.
- The number. The chance that card is the opponent's next draw. It is the unseen copies of that card, divided by the unseen cards. Unseen copies of a card are its blank copies plus any copy I watched go back into the deck. Unseen cards are the opponent's 60 minus every copy I watched go to the inkwell, the discard, or the table, and minus any card revealed in hand. In practice that is the deck plus the unrevealed hand. Two simplifications in that count are listed under Known limitations.

Along with the rows, five counts: cards in the opponent's hand, inkwell, deck and discard, and their lore, the score. These are the numbers players ask each other for at the table. The rules already make the inkwell count public (Comprehensive Rules 8.5.4), and the rest can be counted by looking. Lore is public and must be announced, and players track it with a counter, the companion app, or on paper, so it would be useful if a player in a headset did not have to look away from their opponent for it. In the demo, hand and lore come from the scene file, since the stream shows them; inkwell, deck and discard are counted from the mat positions.

Where the counts sit: in the panel, in a header above the rows. On the mat, in one small strip pinned to the near edge of the opponent's mat, where the two mats meet, labeled with whose counts they are.

Two panel details. Each panel row carries a small thumbnail of the card, since the card is not under the row; on the mat the thumbnail is dropped. The panel keeps the order players already use on deck sites: by ink cost from low to high, and within a cost by card type.

Figure 3 placeholder: Dreamborn screenshot of one of the chosen decks, to be added.

Figure 3. a Dreamborn screenshot of one of the chosen decks, stacked cards with 4x count markers, credited.

4. What I track to fill the rows. Lorcana is a two-player card game, and four of its rules are all the tracking needs.

- A deck holds at least 60 cards.
- A player pays for cards by revealing one card per turn and putting it into a pile called the inkwell, where it is turned face-down at the end of that turn and stays face-down for the rest of the game. Both players see what went in.
- A deck can hold at most four copies of any one card.
- Some card effects make a player show their hand, or put a card back into their deck.

Those four rules are why the engine can stay simple. Two kinds of effect are the exception: they put a card on the table that nobody can name. One puts a card into the inkwell face-down (Judy Hopps, On the Case). The other is Boost, a keyword that puts the top card of the deck face-down under a character. The rules say no player may look at those cards, and a face-down card is not in play (Comprehensive Rules glossary, facedown and under). The tracking records such a card as unknown at its position.

Each pip shows which zone a copy went to, and it is only what I watched happen:

- Inkwell. Seen being revealed and put into the inkwell.
- Discard. Seen going to the discard pile.
- Deck. Seen going back to the deck by a card effect.
- Hand. Seen by a reveal effect, or returned to hand from the table, and not played since.
- Blank. Not seen yet. In the opponent's hand, still in their deck, or placed face-down by an effect such as Boost.

A card on the table is in a sixth zone, in play. It gets no pip, because its row is drawn under it.

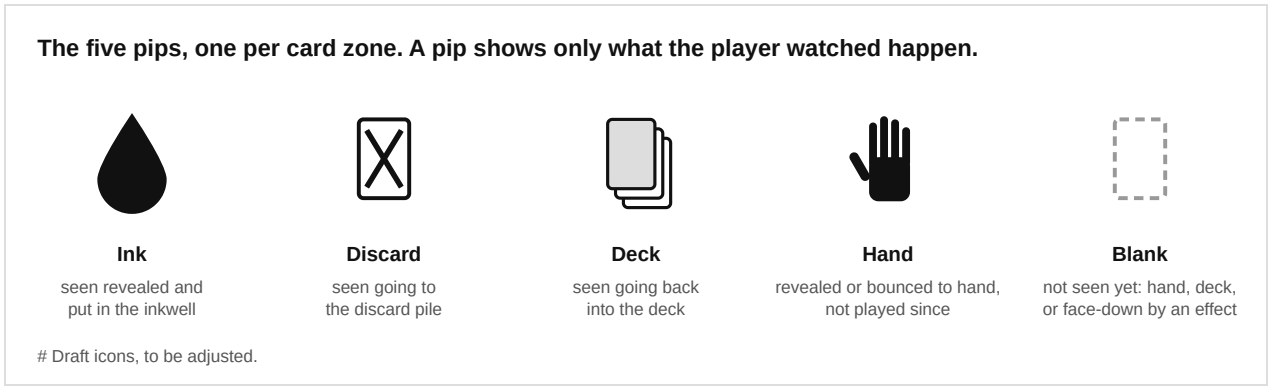


Figure 4. the five pips as five icons in a row, labeled. A drop for the inkwell; card with an X for discard; deck stack; open hand; outlined ghost card for blank.

5. How it gets drawn. The scene file is a text file that holds both decks and every zone change, one turn at a time. In code a scene is a scripted game. I propose five behavior scripts, one class each, plus a handful of small data classes they pass between them. In Unity, the game engine the class builds in, a script is a C# file holding one class, attached to an object in the app. Each class is a noun with one or two verbs. They live in four packages, folders that say where each class belongs: Perception, GameData, Engine and Overlay. The package and class diagrams in the repo show them.

- ScriptedGameLoader (LoadScriptedGame) reads the scripted-game file into GameState once at start, with nothing for anyone to touch. The file is baked into the build, like the HW1 sequence file.
- GameState (NextTurn, PreviousTurn) holds the current turn and applies or undoes its zone changes. It is the hub.
- CountingEngine (BuildOverlayRows) works out the pips and the numbers from GameState. It is the engine, and it is arithmetic. The five counts (hand, inkwell, deck, discard, lore) need no engine; they are read straight from GameState.
- OverlayDrawer (DrawOverlay, PinMat) draws the rows CountingEngine hands it and the counts from GameState, in passthrough, anchored to the mat, the flat playmat the cards sit on. PinMat takes the four corner pinches at setup.
- TurnStepper (OnLaptopCommand) takes the next and back commands from the laptop, then drives one turn: NextTurn or PreviousTurn, then BuildOverlayRows, then DrawOverlay.

The data shapes that pass between them:

- Card. One line of the deck list: a name, how many copies are in the deck list, its type and ink cost.
- MatPosition. A marked spot on the mat, such as play-3 or inkwell-7, with its distance in millimeters from the mat corner.
- CardZoneChange. One copy of one card going from one zone to another. I avoid the word move, because in Lorcana moving means moving a character to a location.
- Turn. One turn of play: its number, its zone changes, and the hand and lore counts at the end of it.
- OverlayRow. What the display draws for one card: its pips, its number, and the positions of any copies in play.

Three short fixed lists sit beside them: CardZone, the six zones a copy can be in; MatArea, the four regions of the mat; and OverlayPlacement, on the mat or on the panel.

In live play, a recognition package would replace ScriptedGameLoader and TurnStepper; GameState, CountingEngine and OverlayDrawer would not change. A machine-learning engine would replace CountingEngine alone.

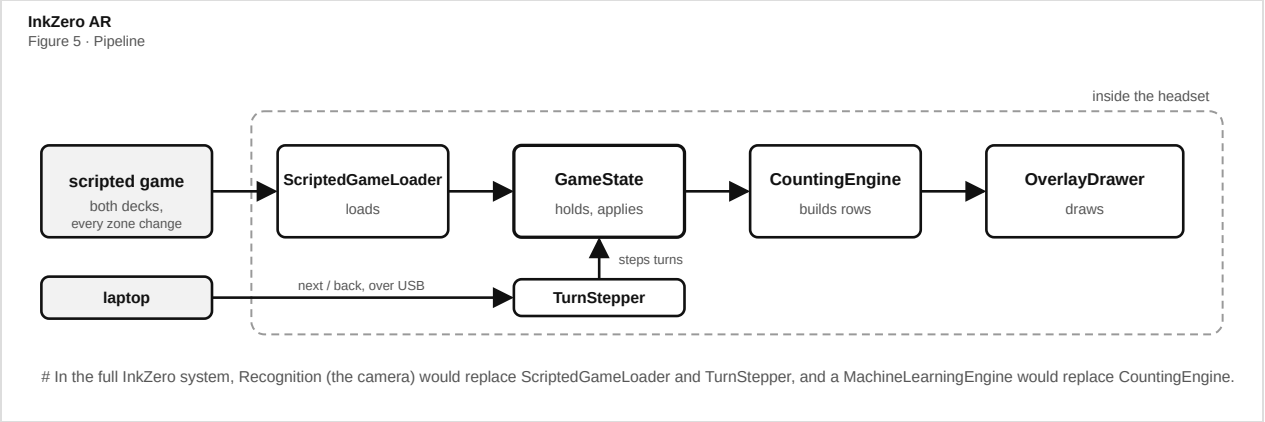


Figure 5. the pipeline. Boxes for the scripted game, ScriptedGameLoader, GameState, CountingEngine, OverlayDrawer, with the laptop feeding TurnStepper. Arrows mark data flow only.

The demo advances turn by turn from the laptop: right arrow for the next turn, left arrow for the one before, sent over a USB cable so there is no wireless link to worry about. The scripted game stands in for the camera.

A zone change is one card going from one zone to another: hand to inkwell, hand to play, play to discard, deck to hand. The scene file is a list of those in order, grouped by turn. Advancing a turn tells GameState to apply the next group, and CountingEngine rebuilds the pips and the numbers.

The headset knows where a card is because I track the mat, not the cards. Lorcana has a loose convention for where things go. From the player's seat, the opponent's inkwell is the far strip, with the face-down cards fanned in a row so they can be counted. The play area is the near strip. The deck and discard sit at the opponent's corner, discard above deck. For this project I track only the opponent's side. I chose to leave the player's own side out so the thing being built stays manageable to define and check; it is on the future-work list.

My mat makes that convention fixed. It is the common 24 by 14 inch playmat size, one per player, with a printed position for each card, like blocking marks on a film set, and the scene file says which card sits in which position on each turn. Sleeved cards are 66 by 91 mm, and a played card can turn sideways, so each play position is 91 mm wide. At that size the mat holds ten fanned inkwell positions, eight play positions in two rows of four, a deck and a discard.

At setup, the player pinches once at each corner of the mat. Those four points pin the mat in space, the way a corner pin places a flat image in After Effects, a video-editing program, and every overlay is measured from them. Setup can take a few touches. My intention is that play is automatic and takes none from the player. I propose to build and check the overlays flat in Unity first, on a photo of the mat, and then put the headset on only for the questions that need it: is the overlay anchored correctly, and are there problems with glare or readability.

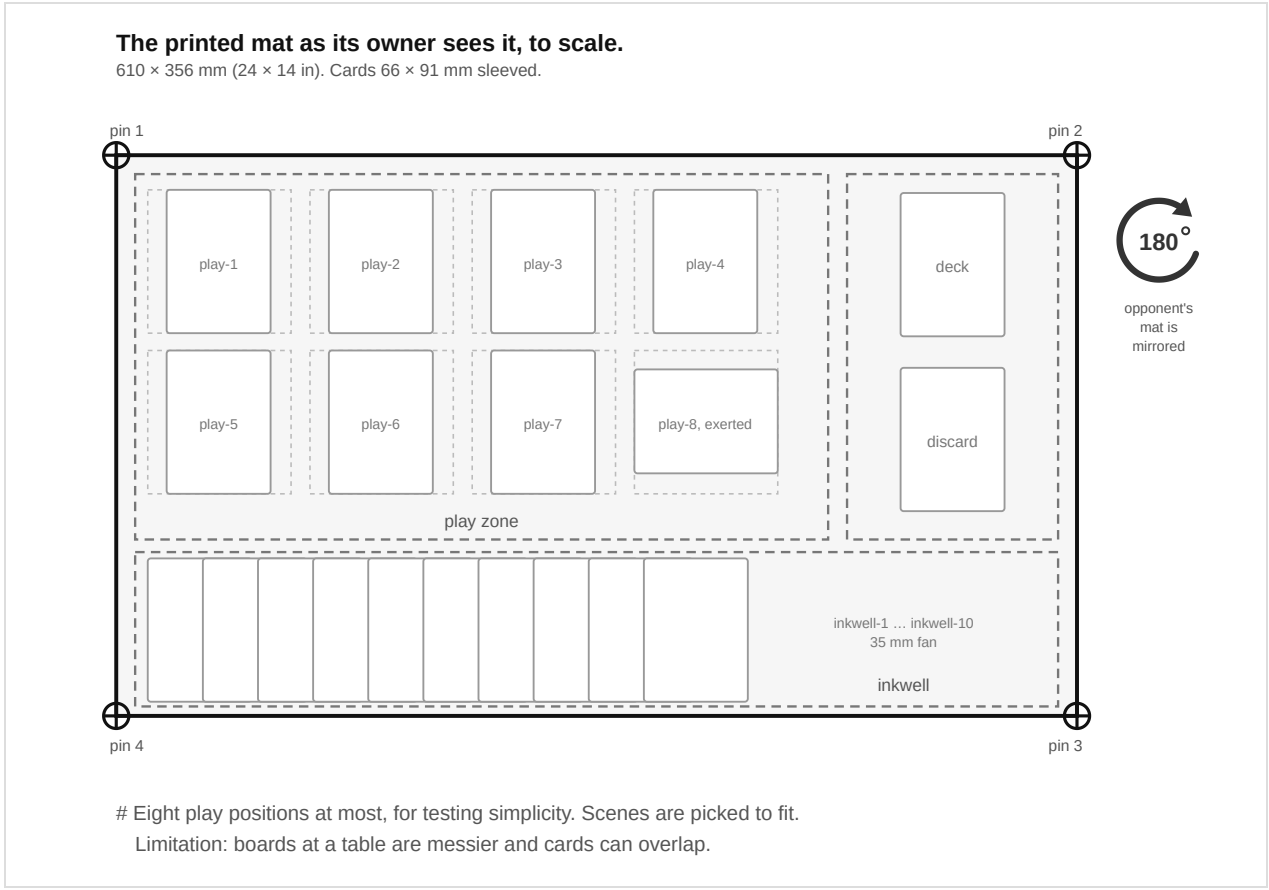


Figure 6. the opponent's mat to scale, 610 by 356 mm, with every position drawn and named and four corner pins.

6. The demo. I propose to stage three scenes on the mat. Each scene is a few turns lifted from a recorded tournament game, with the games still to be chosen. I write down both deck lists and every zone change, turn by turn, the way a shooting script records what happens in a shot. The plays come from a game that was played, so I did not author them, and I am not picking a board that flatters the overlay. If the scenes come from the late rounds of a national event, they are decisions strong players faced, so the overlay is shown next to those choices rather than ones I made up. Whether it changes those choices is the user-study question, and that is future work. I stack both decks to reproduce the recorded draws, which works like setting a seed in a simulation: the same scene plays out the same way each run, and every pip and every number has a known answer to check against. At the checkpoint I step through one scene from the laptop. At the end, all three, in both placements.

How I will test it

The test is the two placements side by side, on the mat and in the panel. I will show one of each, say which I would keep, and say why. The four checks below are how I decide. The first two check the system; the last two check the two placements against each other. Since the assignment puts a user study out of scope, each one is something I can show in the room.

- Correctness. The pips and the numbers match hand-checked answers for every turn of every scene. A wrong pip or a wrong number is worse than none, because a player would act on it. The answers for the numbers come from a precomputed table of unseen copies against unseen cards.
- Latency. Time from the headset receiving a turn command to the frame that draws the updated overlay, in milliseconds. An overlay that lags the table stops being passive; the player either waits for it or ignores it.
- Stability. The on-mat placement is only useful if a row stays under its card. How far the on-mat rows drift off their cards over a ten-minute session at a stated table distance, measured against a ruler placed on the mat in the capture video. Ten minutes is about three scenes. I will adjust it if the scenes run longer or shorter.
- Readability. The second design consideration, checked against the hardware. Whether each overlay reads at a glance, yes or no, against a short checklist. Glare is on the checklist, because tournament cards are sleeved and sleeves reflect light. Recorded on video captured from inside the headset.

The plan is to show correctness and latency on one scene in one placement at the checkpoint on Nov 11, and all four checks on all three scenes in both placements on Dec 7, with the table below filled in.

Check	On the mat	On the panel
Correctness, turns matched out of turns checked		
Latency, ms from command to drawn frame		
Stability, mm of drift over a ten-minute session		
Readability, checklist items passed		

Blank on purpose. To be filled in when the project is complete.

Figure 7. a small table, two columns for on-mat and panel, four rows for the checks, blank until the project is complete.

The test that would answer the question, a study with the headset on versus off, is listed under future work.

Scope for this project

- A passive passthrough overlay on the opponent's side of the mat. During play, the player does not touch it.
- A scene file that holds both decks and every turn, advanced from the laptop.
- For each tracked card, the pips and the number, unseen copies divided by unseen cards.
- Five counts: hand, inkwell, deck, discard and lore.
- The rows and counts drawn on the mat and in a fixed panel.
- Three staged scenes, each a few turns copied from a recorded tournament game, for testing.
- Checkpoint, Nov 11: one scene, rows and counts on the mat.
- Final, Dec 7: three scenes, rows and counts both ways, the four checks written up.

Timeline

- Sprint 1, Oct 20 to Nov 11. Starts by putting the UML diagrams in the repo. UML is a standard way to draw how software is organized. The four here are a use case for the demo, a class diagram for the five behavior classes and the data classes, a package diagram showing how they depend on each other, and a sequence diagram for one turn. They are drafted and will change as the code does. Exit: one scene, rows and counts drawn on the mat, state loaded from a file and advanced from the laptop. Shown live at the checkpoint.
- Sprint 2, Nov 12 to Dec 7. Exit: three scenes, rows and counts on the mat and in a panel, the four checks run and written up.

Assumptions for the build

- A 60-card deck. Sixty is the minimum, competitive players tend to play exactly that to keep their odds up, and larger decks are legal and rare.
- A card gets as many pips as copies in the known list, up to four. The panel has one row per distinct card in the deck.
- The two decks are the ones played in the recorded games the scenes are copied from. Other matchups are future work.
- The opponent's deck list is known. In the demo it comes from the scene file.
- A Meta Quest 3S, the headset the class loaned, in passthrough, built in Unity.
- The opponent's hand is not inferred. A copy is marked in hand only when a card effect revealed it or it was returned to hand from the table. Any copy I have not seen is blank.
- Decks are built from sets released before tokens, a new kind of card piece. Tokens arrived with the set released Oct 16, 2026, and are out of scope for this project.

Known limitations

- The Quest 3S passthrough is low resolution. Whether a player can read the physical card text through it is a question on its own, and it is on the readability checklist.
- The numbers assume the deck is in random order. Some card effects let a player look at or reorder the top of the deck, and the numbers do not account for them.
- A copy seen going back into the deck keeps its deck pip and is counted as unseen, which ignores that it is in the deck for certain and where in the deck it went. The rules say a publicly known card put on the top or bottom of a deck keeps a known order (Comprehensive Rules 8.2.4), so a later engine could use that; the counting engine does not.
- A card placed face-down by an effect (Boost under a character, or an effect-ink) is recorded as an unknown card at that position. It fills the position and leaves the unseen total if it came from the deck, but it cannot be told apart from a card still in the deck, so no pip moves. The scenes avoid Boost decks unless one is needed to show this.
- The readability check is judged by me. It is recorded on video from inside the headset so anyone can disagree.
- The demo is a scripted game. It shows the display, not live tracking of a game as it is played.
- The mat has eight play positions and the scenes are picked to fit. Boards at a table are messier, and cards can overlap.

Positioning and future work

This is a free learning tool for casual play. It is not for sanctioned events, meaning official tournaments. The tournament rules already forbid notes other than lore totals, and they allow electronic devices only when the device is in plain view of both players (sections 5.2 and 5.3), which a headset display is not. The poker and chess communities draw the same line on real-time help. The overlay is visible only to the player wearing the headset. In casual play the opponent knows the tool is there.

No card art is reproduced; the overlay draws marks and numbers. Ravensburger, the company behind Lorcana, publishes guidelines for community use, and the project follows them and carries the statement they ask for.

Future work, in the order it would come:

- Card recognition, so the camera advances the state instead of the script.
- Deck-list inference. In live play the opponent's list would be inferred from the first cards played, narrowed against the current top decks. An inferred card would be drawn faded and a confirmed card solid. Every inferred card would be assumed at four copies, the worst case.
- A viewer version of the panel, for a stream or for judges, drawn from the same scene state.
- The same rows and counts for the player's own side. The counts strip is already labeled by side, so it would become a double strip, and the panel would gain a second column or a twin on the other side of the mat.
- A setup-time choice of panel order, picked before the first turn so the overlay stays untouched during play. The default is by ink cost. The other choice is by the odds, so the most likely next draw sits at the top and the rows reorder as the game moves.
- A readout for the chance within the next two or three draws, since some effects draw more than one card.
- A machine-learning engine that infers the opponent's likely next plays from their deck and the board. The display built in this project is the surface that engine would feed.
- A study with the headset on versus off, two fixed decks, sides swapped, best of three.
- Display glasses are the device this tool wants to end up on. The Ray-Ban Display is a heads-up panel with no world tracking, so of the two designs here, the panel is the one that would port to it. Meta's VR glasses are due in spring 2027, with no developer details as of this writing.

References

- Kadish, D., Sarkheyli-Hägele, A., Font, J., Niehorster, D. C., and Pederson, T. (2022). Towards situation awareness and attention guidance in a multiplayer environment using augmented reality and Carcassonne. arXiv:2208.09094.
- Gamba, A., and Monroy-Hernández, A. (2023). PokAR: Facilitating poker play through augmented reality. arXiv:2301.00505.
- Cioppa, A., Verdonck, A., Henry, M., Van Droogenbroeck, M., and La Rocca, R. (2026). TCG-AR: Real-time multi-view augmented reality for trading card game streaming. University of Liège. arXiv:2607.02090.
- Szalavári, Z., Eckstein, E., and Gervautz, M. (1998). Collaborative gaming in augmented reality. Proceedings of VRST '98, pp. 195–204.
- Willett, W., Jansen, Y., and Dragicevic, P. (2017). Embedded data representations. IEEE Transactions on Visualization and Computer Graphics, 23(1), 461–470.
- Ravensburger (2026). Disney Lorcana Challenge, official broadcast overlay, 2026 season stream.
- Ravensburger (2026). Disney Lorcana TCG Tournament Rules, 7.14.2026 update. Sections 1.3.1.1, 5.2, 5.3, 5.5.
- Ravensburger. Disney Lorcana Comprehensive Rules (sections 4.3.3, 7.1.10, 8.1 to 8.7, 9.1), and Whispers in the Well release notes (Boost keyword; Judy Hopps, On the Case).